DÉJÀ VIEW

Personal Entertainment Intelligence Platform

Consumer | AI Recs | Social

Project Blueprint · Architecture · Tech Stack · Roadmap

Watch History

Smart personal movie journal

Recommendations

AI-powered personalized picks

Discovery Feed

Daily entertainment intelligence

Platform Overview

Déjà View solves decision fatigue in entertainment through personal history, memory, and intelligence.

Watch History

Consumer

- Save movies, shows & anime
- Personal ratings & reviews
- Rewatch tracking
- Smart journal interface

Recommendations

Consumer + Social

- AI taste modelling
- Hidden gems discovery
- Because-you-liked-X logic
- Trending & mood picks

Discovery Feed

Consumer + Social

- Upcoming release news
- Trailer alerts
- Awards & casting updates
- Daily engagement hook

SYSTEM ARCHITECTURE

DATA SOURCES
- TMDB API
- User Uploads
- News RSS
- Streaming APIs

›

PROCESSING
- Preference Engine
- Rec Algorithms
- NLP Tagging
- Feed Curation

›

BACKEND
- FastAPI Server
- Celery Workers
- Redis Cache
- PostgreSQL

›

DELIVERY
- REST API
- WebSocket
- Web App
- Mobile App

Recommendation Engine

01 **Data Collection**
Ratings, watch history, genre affinity, rewatch signals, and explicit favourites are aggregated per user.

02 **Preference Modelling**
Collaborative filtering + content-based signals build a latent taste vector per user, updated incrementally.

03 **Candidate Generation**
Top-N candidates pulled from TMDB catalogue filtered by language, era, genre, and streaming availability.

04 **Re-ranking**
Candidates scored by novelty, recency, social signals, and mood context. Diversity injection prevents filter bubble.

05 **Personalised Feed**
Final ranked list delivered via REST. WebSocket push for real-time updates on new releases matching taste.

ALGORITHM VARIANTS

Collaborative Filtering
Users with similar taste patterns recommend hidden gems you haven't seen yet.

Content-Based
Genre, director, cast, and thematic tags matched to your watch history vectors.

Hybrid Ensemble
Weighted blend of CF + CB with contextual re-ranking and mood injection.

Trending Boost
Viral picks amplified by recency decay curve — freshness without noise.

App Dashboard Mockup

Tech Stack & API Design

Frontend (React)

React 18 + Vite
SPA framework

Zustand
State management

Recharts
Taste analytics

Framer Motion
UI animations

Socket.IO
Live feed updates

Tailwind CSS
UI styling

Backend (FastAPI)

FastAPI
Async REST + WS

Celery + Redis
Async rec jobs

SQLAlchemy
ORM layer

PostgreSQL
Primary database

JWT / OAuth2
Auth

Alembic
DB migrations

Intelligence Layer

TMDB API
Movie/show metadata

scikit-learn
Rec algorithms

NumPy + Pandas
Data modelling

Redis Cache
Low-latency recs

Sentence-BERT
Semantic tagging

NewsAPI
Entertainment feed

Infrastructure

Docker Compose
Local dev

AWS ECS
API workers

CloudFront
Static CDN

S3
Poster/image store

Grafana
Internal metrics

Sentry
Error tracking

API ENDPOINTS

GET	/api/user/{id}/watchlist	Full watchlist with status, ratings, and filters
GET	/api/user/{id}/history	Watch history with timestamps and rewatches
GET	/api/user/{id}/recommendations	Personalised top-N picks with explanation
POST	/api/user/{id}/watch	Log a new watch entry with rating and review
GET	/api/movies/{id}	Full movie detail: cast, trailer, streaming info
GET	/api/feed/entertainment	Latest trailers, news, and release alerts
GET	/api/discover/nearby-taste	Social: users with similar taste nearby
WS	/ws/user/{id}/feed	WebSocket: real-time rec and feed push

Database Schema

users

id	UUID PK
username	VARCHAR
email	VARCHAR
avatar_url	TEXT
plan_type	ENUM
created_at	TIMESTAMPTZ

movies

id	UUID PK
tmdb_id	INT UNIQUE
title	VARCHAR
genres	JSONB
poster_url	TEXT
runtime_min	INT

watch_entries

id	UUID PK
user_id	UUID FK
movie_id	UUID FK
status	ENUM
rating	FLOAT
review	TEXT
watched_at	TIMESTAMPTZ

watchlist

id	UUID PK
user_id	UUID FK
movie_id	UUID FK
mood_tags	JSONB
priority	INT
added_at	TIMESTAMPTZ

recommendations

id	UUID PK
user_id	UUID FK
movie_id	UUID FK
score	FLOAT
reason	TEXT
created_at	TIMESTAMPTZ

rewatch_log

id	UUID PK
entry_id	UUID FK
rewatched_at	TIMESTAMPTZ
new_rating	FLOAT
notes	TEXT

Roadmap & Go-To-Market

Phase 1
Foundation
Wks 1–4

- FastAPI skeleton + PostgreSQ
- TMDB API integration
- User auth (JWT + OAuth2)
- Basic movie search + detail
- Docker Compose dev setup

Phase 2
Watch System
Wks 5–8

- Watch history CRUD
- Personal ratings & reviews
- Rewatch tracking
- Watchlist with mood tags
- Redis caching layer

Phase 3
Rec Engine
Wks 9–12

- Collaborative filtering model
- Content-based tagging
- Because-you-liked-X feed
- Streaming availability filter
- Rec explanation text

Phase 4
Engagement
Wks 13–16

- Entertainment news feed
- Trailer + release alerts
- WebSocket push updates
- React web dashboard
- Mobile-first responsive UI

Phase 5
Social + Growth
Wks 17–20

- Friend watchlists + sharing
- Yearly taste recap
- AI mood-based suggestions
- Public API for third parties
- Streaming service OAuth

BUSINESS MODEL

Consumer — Free Tier

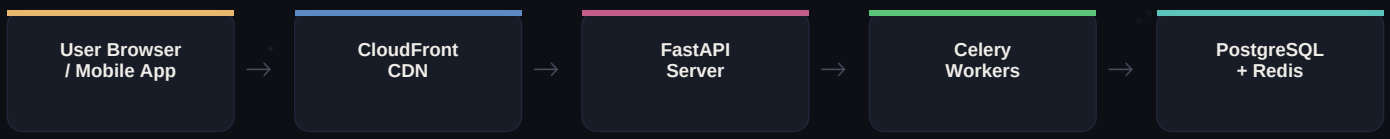
- Full watch history & watchlist
- Basic recommendations
- TMDB movie discovery
- Entertainment news feed

Consumer — Premium £3.99/mo

- Real-time rec push alerts
- Advanced taste analytics
- Friend sharing & social feed
- Yearly Wrapped-style recap
- Mood AI + randomiser

Infrastructure, Privacy & Setup

DEPLOYMENT ARCHITECTURE



PRIVACY & DATA ETHICS

No biometric data stored
User content is ratings, reviews, and preferences only — no browsing or third-party tracking.

GDPR compliant by design
Data export and account deletion endpoints built from day one. Consent flows for all features.

TMDB data only — no scraping
All movie metadata from TMDB API under their terms. No scraped studio data.

OAuth2 for third-party auth
Google/Apple sign-in options. Passwords bcrypt-hashed. Sessions JWT-scoped.

QUICK START

```
# 1. Clone repository
git clone https://github.com/your-org/deja-view && cd deja-view

# 2. Start services (Docker)
docker-compose up -d   # PostgreSQL + Redis + FastAPI + Celery

# 3. Install deps & migrate
pip install -r backend/requirements.txt && alembic upgrade head

# 4. Seed TMDB genres
python scripts/seed_genres.py --api-key $TMDB_API_KEY
```